vesuvius-challenge-surface-detection

9th Place Solution SUPER-CONSERVATIVE(0.623->0.616)

Rank	9
Team	阿對對對對隊
Author	tingyi
Topic ID	679241
Version	Original

> This is my first time participating in a competition, so please bear with me if my solution write-up isn't perfect.

Thank you to the organizers for hosting a competition where we could fully unleash our creativity. I would also like to thank the Competition Grandmasters in the discussion forums for enthusiastically sharing their solutions. Some of my methods were adapted directly from the discussions. Although some of them ultimately didn't work out, I still learned a lot of highly valuable things.

Equipment requirements

- We used 9800X3D + 64GB RAM + RTX 5090 (by TINGYI).

Model Training

- Trained the baseline model (`nnUNetTrainerMedialSurfaceRecall`) provided by the organizers using YOLO26's MUSGD.
- Patch Size was set to 192. Increasing it to 224 or 256 improved the Dice score but caused the Topo score to drop.
- Used Group Norm (group = 32) and GELU (not necessarily helpful; increases VRAM usage).

Inference

- Single Fold + TTA + tile size 0.4

Post-processing

1. Remove Small 6-Connected Components (< 5000)

Connectivity	Final Score	Surf Dice	VOI Score	Topo Score
None	0.5968	0.8800	0.5680	0.2998
6	0.6096	0.8799	0.5682	0.3426
18	0.6029	0.8799	0.5682	0.3202
26	0.6027	0.8799	0.5682	0.3196

2. Line Normalization

To reduce b1 errors caused by thin sheet predictions, a line normalization step is applied:

1. For each 2D slice along the Z-axis, apply skeletonize to extract the centerline of thin paper structures.
2. Reconstruct via EDT, expanding the skeleton back to a uniform thickness of radius `r=1.5`.
3. The volume is split into $8 \times 160 \times 160 \times 160$ blocks; pixels within border ≤ 8 of each block are left untouched to avoid artifacts near ignore mask boundaries.
4. The reconstructed result is OR-ed back into the original prediction (additive, not replacement).

This ensures extremely thin paper sheets are thickened without removing any existing predictions, reducing topological breaks along fragile thin regions.

Radius	Final Score	Surf Dice	VOI Score	Topo Score
Only <5000	0.6096	0.8799	0.5682	0.3426
1px	0.6132	0.8800	0.5682	0.3545
1.5px	0.6132	0.8801	0.5669	0.3558
2px	0.6085	0.8769	0.5637	0.3475

---

3. Gap Filling & Diagonal Repair

Thin paper structures often have small holes or diagonal breaks in the prediction mask. A two-stage morphological repair is applied — and critically, the entire process is run in a loop. Experiments show a linear improvement in score from iteration 1 through iteration 5, making repeated application strongly beneficial.

Stage 1 — Gap Filling

The key idea: "if both sides of a gap are solid, fill the gap in between."

- For each axis, precompute a support map — a pixel is a reliable anchor only if it is already predicted positive and has at least `min_neighbors` neighbors in the same 2D plane.
- Scan across each axis: if two anchor pixels are separated by a gap of $\leq$ `max_gap` voxels, the voxels in between are OR-ed to 1.
- This reliably bridges small breaks without over-filling, since both endpoints must independently pass the support check.

Stage 2 — Diagonal Fill

Axis-aligned filling misses diagonal breaks (e.g., a sheet running diagonally in XZ or YZ). This step applies the same sandwich logic along diagonal directions using small $3 \times 3 \times 3$ kernels with two opposing off-center taps. If both diagonal neighbors of an empty voxel are positive, the voxel is filled.

> ☒terative Gain: Each additional pass of Gap Filling + Diagonal Fill continues to close newly exposed breaks revealed by previous iterations, yielding consistent linear score gains up to at least 5 iterations.

Iteration	Final Score	Surf Dice	VOI Score	Topo Score
RM<5000 + LN	0.6132	0.8801	0.5669	0.3558
1	0.6199	0.8796	0.5672	0.3784
2	0.6239	0.8795	0.5672	0.3918
3	0.6248	0.8794	0.5673	0.3950
4	0.6258	0.8793	0.5673	0.3983
5	0.6264	0.8792	0.5673	0.4005
6	0.6264	0.8791	0.5673	0.4004

4. Gaussian Filtering

Apply a Gaussian filter (`sigma=1`) to the boolean mask, then threshold back to `bool` via `> 0.5`. This trick turned out to be surprisingly effective, though the exact mechanism is not entirely clear.

Method	Final Score	Surf Dice	VOI Score	Topo Score
RM<5000 + LN+SW	0.6264	0.8792	0.5673	0.4005
RM<5000 + LN+SW+GU	0.6323	0.8771	0.5668	0.4232

5. Second Removal of Small 6-Connected Components (< 5000)

6. Block-wise Small Component Removal

Divide into 8 blocks following the TOPO algorithm, then remove 6-connected components < 10.

7. 2×2 Diagonal Bridging

Proposed as a targeted response to this discussion.

The method specifically detects the pattern shown below and fills the entire area if detected. It is highly effective and carries almost zero risk of dropping the score for any given case.

Method	Final Score	Surf Dice	VOI Score	Topo Score
RM<5000 + LN+SW+GU	0.6323	0.8771	0.5668	0.4232
RM<5000 + LN+SW+GU+2x2DG	0.6337	0.8771	0.5668	0.4277

Unused Post-processing Methods

1. Separating Touching Objects / De-adhesion

(Based on @hengck23's killer ant method. Hard to see score improvements from this component, but it remains critical for topological correctness.)

To balance computational efficiency and segmentation accuracy, the algorithm uses a block-wise (8-slice block) iterative mechanism with six core steps.

Step 1 — 2D Ray-Casting Collision Detection

Within 2D slices along the Z-axis, PCA computes the local normal direction at each object's edge. Virtual rays are cast along these normal vectors. If a ray passes through background and collides with another high-confidence prediction, the location is flagged as a potential adhesion point.

Step 2 — High-Confidence Target Voting

To handle simultaneous multi-object adhesion:

- All collision pairs are aggregated globally.
- The two high-confidence (> 0.9) predicted 6-connected components most frequently co-occurring in 3D space are identified.
- The pair with the highest vote count is selected as the sole cutting target for the current iteration.
- Remaining adhesions are deferred to subsequent iterations.

Step 3 — Shortest Path & Midpoint Sampling

Once the adhesion pair is established:

- The shortest path between the ray's start and collision point is computed.
- The midpoint of this path is designated as the adhesion boundary.
- A sparse sampling strategy selects only 3 representative collision points, dramatically improving throughput.

Step 4 — Geodesic Basin & Marker-Controlled Watershed

Using the 3 midpoints from Step 3 as seeds:

- BFS computes the 3D geodesic distance to adjacent connected components.
- The negative of these distances forms the topographic basin for the watershed.
- The two high-confidence components from Step 2 serve as initial markers, guiding watershed flows to converge within the basin and precisely delineating the 3D separation plane.

Step 5 — Boundary Cleanup & Gap Generation

- Morphological dilation is applied around the identified segmentation interface.
- A physical gap of ~2 pixels wide is forcibly removed to guarantee complete disconnection.

Step 6 — Iterative Processing

Steps 1–5 repeat until either:

- No new valid cutting points are found, or
- The pre-set maximum iteration count is reached.

⚠ Methodological Limitations

Limitation 1: Seed Adhesion Dependency

This algorithm relies heavily on `high_conf_labels` from the neural network as watershed anchor points. If the model's high-confidence predictions are already merged (i.e., two distinct objects predicted as a single connected component), the Step 2 voting mechanism cannot distinguish them — rendering the algorithm ineffective in that region.

Limitation 2: Hole Generation at Boundaries

The fixed ~2-pixel removal in Step 5 is a blunt instrument. While it reliably severs the adhesion, the rigid width is not adaptive to local structure thickness. In practice, this hardcoded gap can introduce more holes than it resolves.

---

2. 2D Slice Line Interpolation / Completion

Step 1 — Skeletonization

All 2D slices (both Z-axis and Y-axis) are skeletonized before processing. This ensures they can be jointly handled during the later Line Norm stage.

Step 2 — Endpoint Detection

Using a 3×3 kernel, we scan each skeletonized slice. A pixel is classified as an endpoint if its value is `1` and exactly one of its eight neighbors is also `1`.

Step 3 — Tangent Estimation via PCA + DFS

For each detected endpoint, we perform DFS backtracking along the skeleton to collect its local neighboring pixels, then apply PCA on these points to estimate the tangent direction — determining the orientation the endpoint is pointing toward.

Step 4 — Endpoint Pairing

After collecting all endpoints and their PCA-estimated tangent angles, we perform endpoint pairing. A pair is considered valid if both of the following conditions are met (thresholds are tunable):

- The angle between the connecting line and both tangent directions is $< 45°$
- The Euclidean distance between the two endpoints is ≤ 40 pixels

Step 5 — Skeleton-level Connection

For each valid endpoint pair, we directly draw a line on the skeleton to bridge the gap between the two endpoints.

Step 6 — Line Norm

Once all connections are completed, the repaired skeleton is passed into the Line Norm pipeline. This process is applied independently on both the Z-axis and Y-axis slices.

⚠ Methodological Limitations

Limitation : Randomness

- In certain cases, when model predictions are already poor, applying this patch will make matters worse. (PB's situation)

Due to the negative feedback from PB, we abandoned many CV-boosting methods until after the competition ended, when we uploaded the highest CV version for testing.

CV	Final Score	Surf Dice	VOI Score	Topo Score	
RM<5000 + LN+SW(kernel=3)+GU+2x2DG	0.6337	0.8771	0.5668	0.4277	Competition Submission Version
RM<5000 + LN+SW(kernel=5)+GU+2x2DG+ZLine+YLine	0.6391	0.8757	0.5672	0.4468	Submit the version after the competition

Supplemental Q&A

> Tom · 2026-02-28T04:34:49.370000
>
> ```
> The Patch Size was set to 192. Although increasing it to 224 or 256 improved the Dice score, it actually caused the Topo score to drop.
> ```
>
> Congrats team! I wonder do you experiment smaller patch sizes? In my exp, smaller patch size can increase topo and voi but lowering the surface. I think large context would lead more missing components.

> tingyi · 2026-02-28T05:04:06.307000
>
> In the very early stages of the competition, I experimented with patch sizes of 96, 128, 160, 192, and 224. At the time I was using the original nnUNet, so performance appeared to saturate around 160. However, what I observed was that scaling from 96 to 192 showed linear growth in both Topo and VOI metrics. That said, the evaluation code I was using back then wasn't the official one — it was something someone had written on the forum — and it also didn't account for the influence of labels containing b1 and b2. So I'm not entirely confident that experimental data was accurate, especially since I never re-ran those experiments later.

Team identifier: 阿對對對對隊（チーム名）